



DonChiDon

經天一起動吃動

目錄

- 隊伍介紹
- 創作理念
- 成果說明
- 程式說明



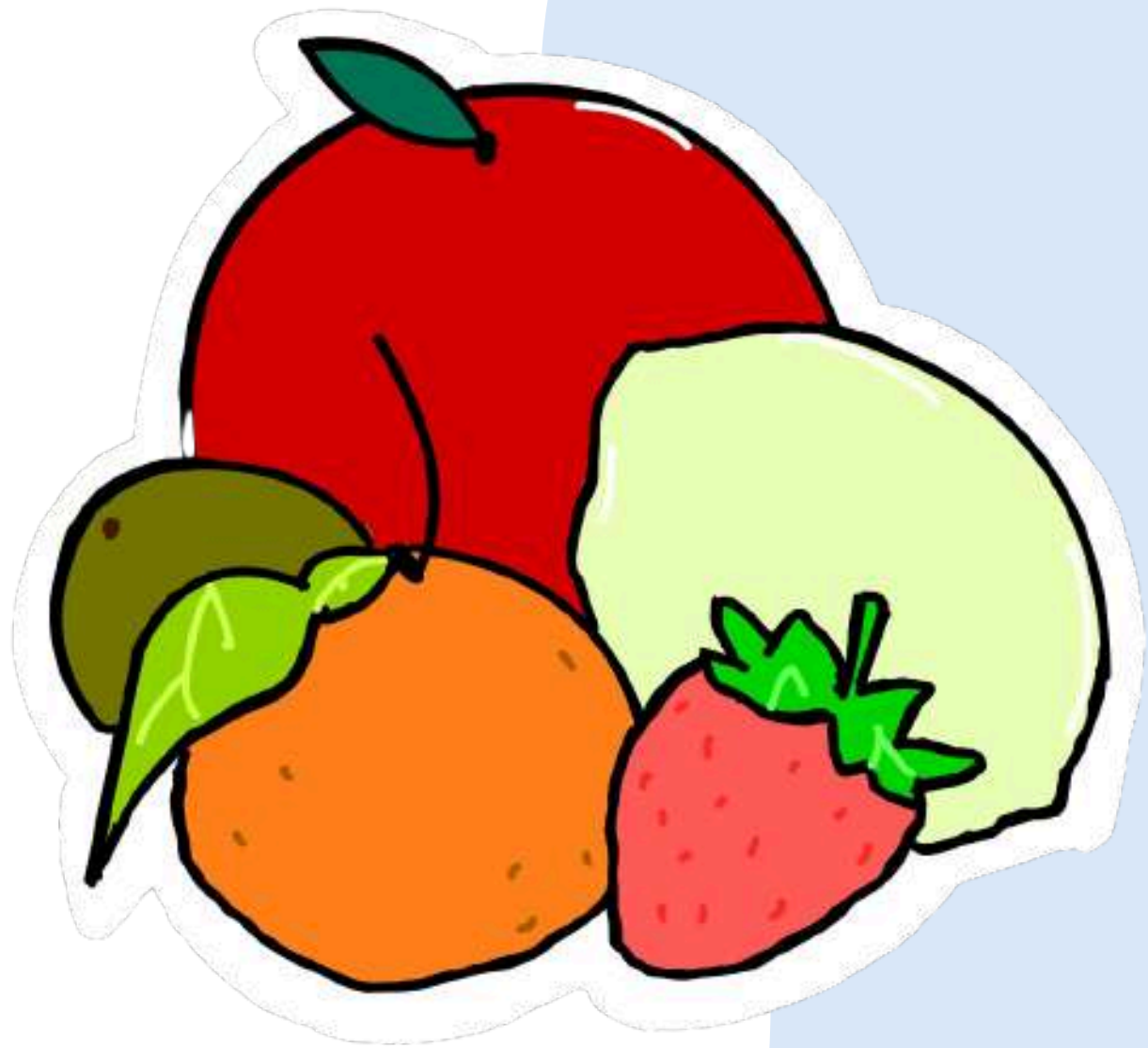
隊伍介紹

我們是「經天一起動吃動」，團隊成員有關蕎蓁、王乙軒、林冠綺、劉庭秀、李維宸，所有成員皆來自經濟系。我們從一年級開始就對程式設計抱有高度興趣，並皆修習過程式設計相關課程，且持續進修，企圖提高程式設計相關能力，也希望能透過此次參賽，檢視並提升自我能力，與建立彼此團隊合作之默契。

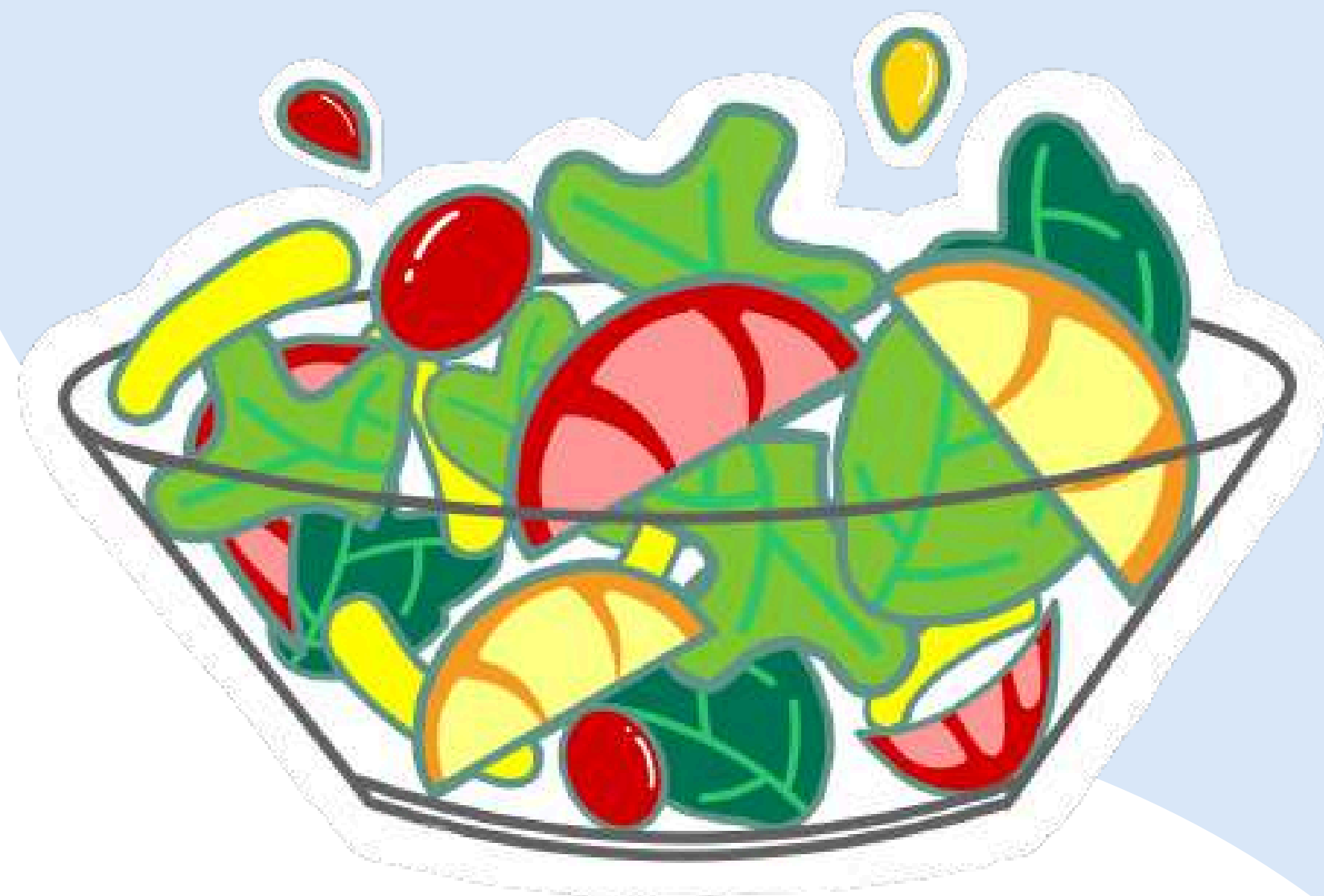


創作理念

升上大學後，身體在久坐不動和可口美食的雙重壓力之下，便漸漸臃腫起來。而我們也發現因為生活型態的改變導致社會上也有許多人有相同困擾，所以我們想解決此煩惱。在控制體態的過程中，計算自己吃了多少卡路里、需要多少運動是十分複雜而麻煩的，也讓不少人因此望之卻步。因此，我們希望能設計一款簡單而直覺的小工具，並加入一些其他應用程式沒有的功能，來讓控制體態變得更簡單輕鬆！



成果說明



個人資料頁面

個人資料頁面

分析結果頁面

食物紀錄頁面

運動紀錄頁面

數據總結頁面

個人資料

身高 (cm)

體重 (kg)

年齡

性別 ☐ 男 ☐ 女

目標 ☐ 減肥 ☐ 維持 ☐ 增肥

期望活動量 ☐ 輕度 ☐ 中度 ☐ 重度



首次打開檔案時會要求使用者輸入身高與體重等基本資料，用以計算一天所需的卡路里，並依照所選之目標及期望活動量，為不同使用者制定專屬目標熱量。完成後可以透過左邊按鍵來記錄攝取的食物、運動量或查看數據總結與分析結果。

個人資料頁面

個人資料

Error

出現錯誤，請檢查以下可能情況:
1. 若為初次使用或是重新啟動後，確認每個欄位皆已填寫
2. 身高、體重、年齡均填寫數字

數據總結頁面

身高 (cm)

體重 (kg)

年齡

性別 ☐ 男 ☐ 女

目標 ☐ 減肥 ☐ 維持 ☐ 增肥

期望活動量 ☐ 輕度 ☐ 中度 ☐ 重度



為了防止使用者輸入錯誤或沒有輸入就點選其他頁面等情形，製作了**Error**視窗來提醒輸入錯誤的使用者

分析結果頁面



在分析結果頁面，程式透過輸入的基本資料，簡單計算出基礎代謝率、目標熱量、BMI以及體重等級

食物記錄頁面

個人資料頁面

分析結果頁面

食物紀錄頁面

運動紀錄頁面

數據總結頁面

食物紀錄

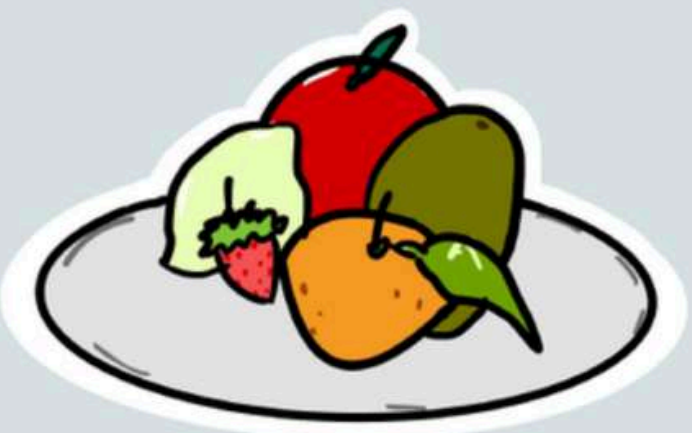
吃了

☐ 早餐 ☐ 午餐 ☐ 晚餐 ☐ 其他

熱量(kcal, 自由選填):

ok

* 若未按下此按鈕將不會進行記錄



記錄攝入的食物時，使用者需輸入食物名稱與種類（早餐、午餐等）。若使用者未填寫卡路里，程式會利用我們所建立的資料庫，搜尋該食物的卡路里，若沒有完全符合的名稱，我們會計算相關度，來找出最接近該食物的卡路里。使用者也可自行填寫食物的熱量，更準確地紀錄攝取的卡路里。

食物記錄頁面



同樣的，若有輸入錯誤等問題，會出現Error視窗提醒使用者。

運動記錄頁面

個人資料頁面

分析結果頁面

食物紀錄頁面

運動紀錄頁面

數據總結頁面

運動紀錄

項目

足球

運動時間(min):

30

選擇強度/類型

* 若未按下此按鈕將不會進行記錄

休閒足球

足球比賽

繼續輸入

重填運動項目/時間





紀錄運動時，使用者需輸入運動項目與時間，並用下拉選單選擇運動強度。按下「繼續輸入」可以再紀錄新的運動，如果輸入錯誤也可以按「重填運動項目/時間」來修正。

數據總結頁面

數據總結

目標熱量1713kcal - 攝取熱量2555kcal + 消耗熱量180kcal = 剩餘熱量-661kcal
~ 攝取超過了！多運動吧 ☹ ~

-----以下為您的細項-----

個人資料頁面

分析結果頁面

食物紀錄頁面

運動紀錄頁面

數據總結頁面

攝取食物	熱量
早餐	42 kcal
燕麥奶	42
午餐	1000 kcal
火鍋	1000
晚餐	1200.0 kcal
珍珠奶茶	600.0
水果	600.0
其他	313 kcal
霜淇淋	139
義美小泡芙	174

運動消耗	180.0 kcal
跑步	180.0

關閉重新啟動

刪除特定紀錄

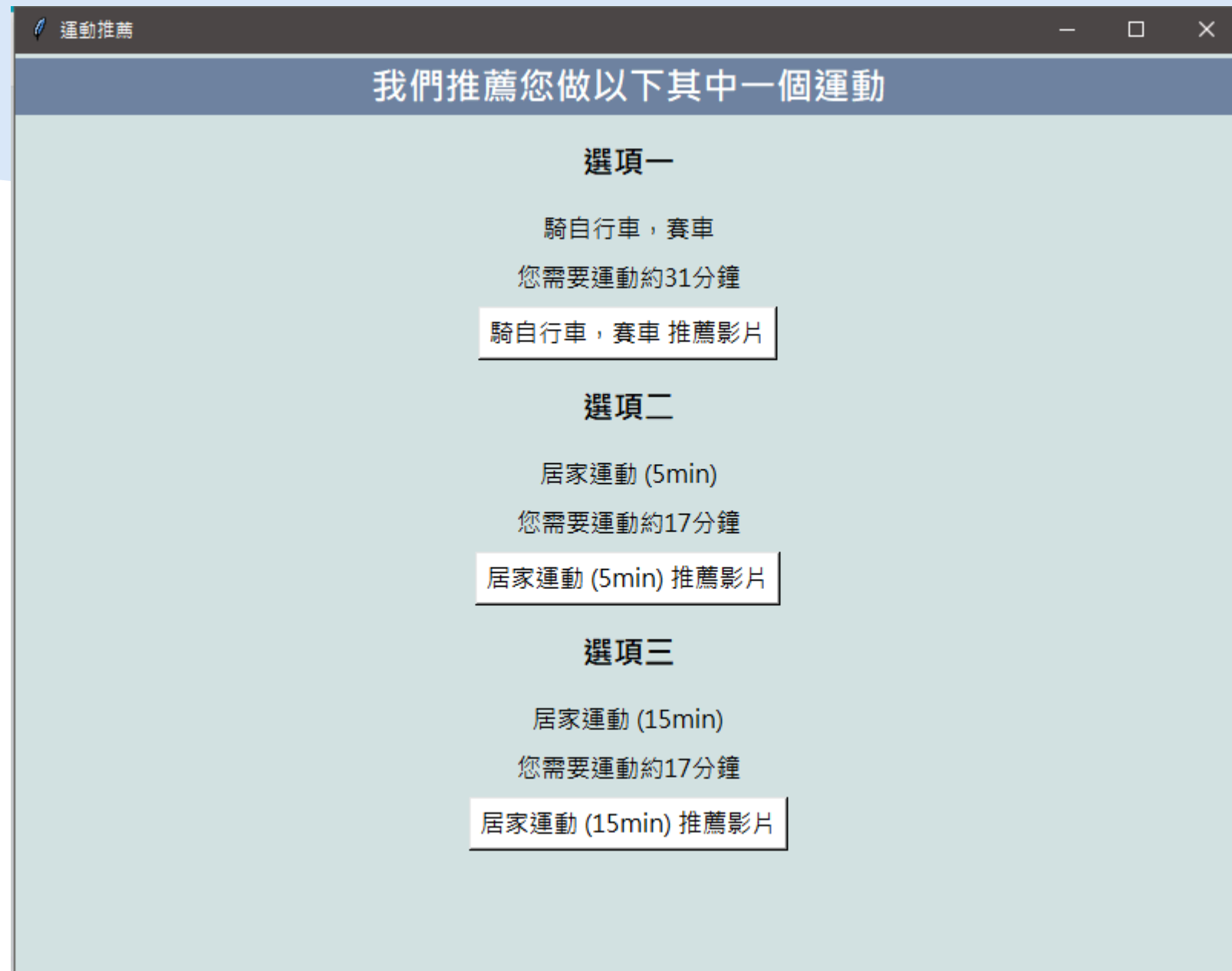
刪除所有記錄

運動推薦



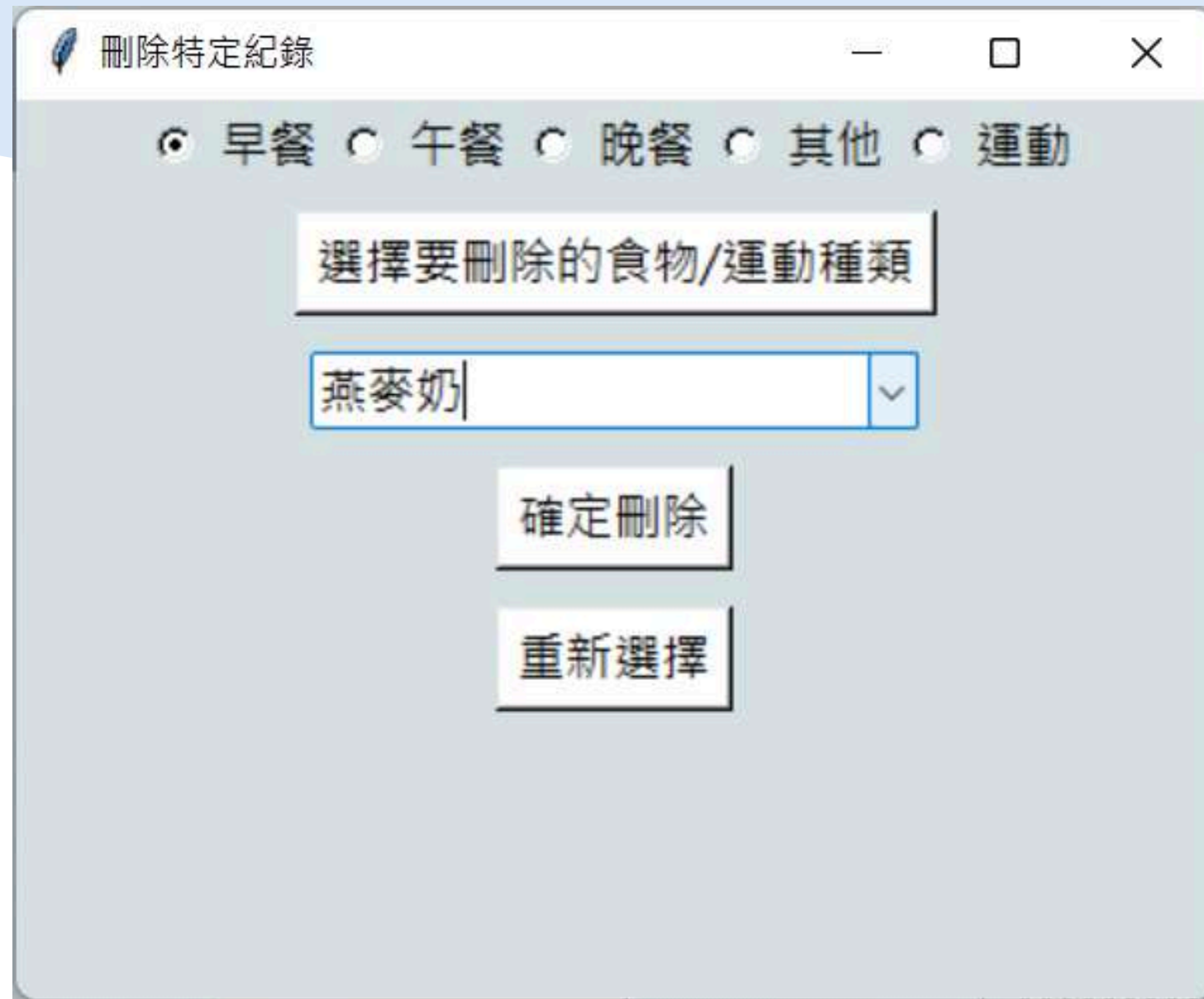
在數據總結頁面中，使用者可以看到自己每餐分別吃了什麼、做了什麼運動，以及其各別所占卡路里，也可以得知與目標之差距。程式關閉後都會自動儲存紀錄，使用者也能在此頁面選擇一鍵刪除所有紀錄或是重啟程式。

運動推薦頁面



透過數據總結頁面右下角的"運動推薦"按鈕進入運動推薦頁面。在這個運動推薦小視窗中，程式會根據當前消耗卡路里與目標卡路里之差距推薦三項運動。有些推薦運動備有youtube連結，此時點選「推薦影片」按鈕會開啟影片網址。如果使用
者攝取熱量減去消耗熱量小於目標熱量，程式會輸出使用者需運動的時長，反之則輸出「運動時間自訂」

刪除特定記錄頁面



The screenshot shows a dialog box titled "刪除特定紀錄" (Delete Specific Record). At the top, there are five radio buttons for selection: "早餐" (Breakfast), "午餐" (Lunch), "晚餐" (Dinner), "其他" (Other), and "運動" (Exercise). Below these, a label "選擇要刪除的食物/運動種類" (Select the food/exercise type to delete) is positioned above a text input field. The input field contains the text "燕麥奶" (Oat milk) and has a dropdown arrow on its right side. At the bottom of the dialog, there are two buttons: "確定刪除" (Confirm Delete) and "重新選擇" (Re-select).

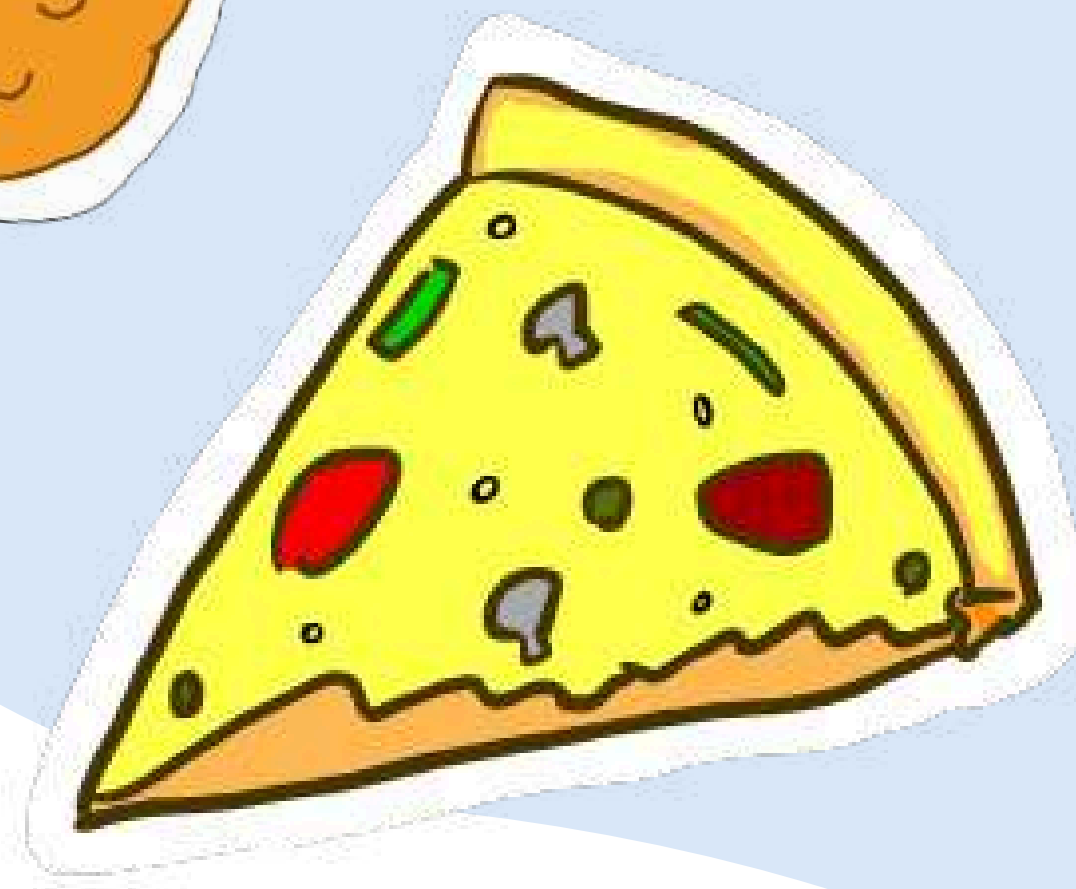
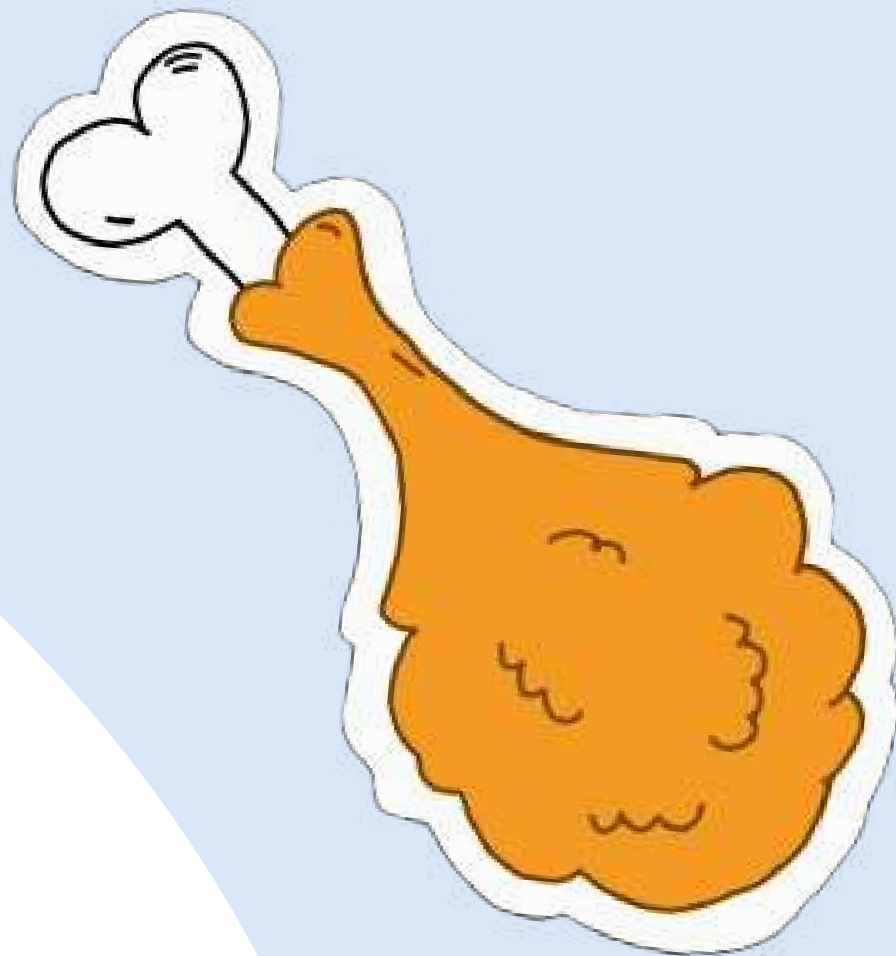
透過數據總結頁面右方之「刪除記錄」按鈕進入刪除紀錄頁面。在刪除記錄小視窗中，使用者可以選擇想刪除的食物或運動，並按「確定刪除」進行刪除。

總論

- 應用性：能自動儲存並估算攝取與消耗的熱量，解決熱量紀錄不易的問題。
- 創意性：除了基本的紀錄熱量與目標熱量之計算外，還增加客製化推薦運動功能，達到獨特性。
- 挑戰性：使用tkinter製作介面、Prettytable製作表格、pandas讀取資料庫資料、寫檔讀檔功能，以及有邏輯性的撰寫程式碼等，運用許多python技巧。
- 完成性：這份程式提供使用者自由選擇是否輸入食物熱量，紀錄後能選擇刪除特定紀錄或是一鍵刪除所有紀錄。第一次輸入個人資料後隨時都能回到頁面進行修改，並且在關閉程式後能自動保存數據。另外，此程式具有完善的防呆裝置，避免使用者操作錯誤使程式無法運行。

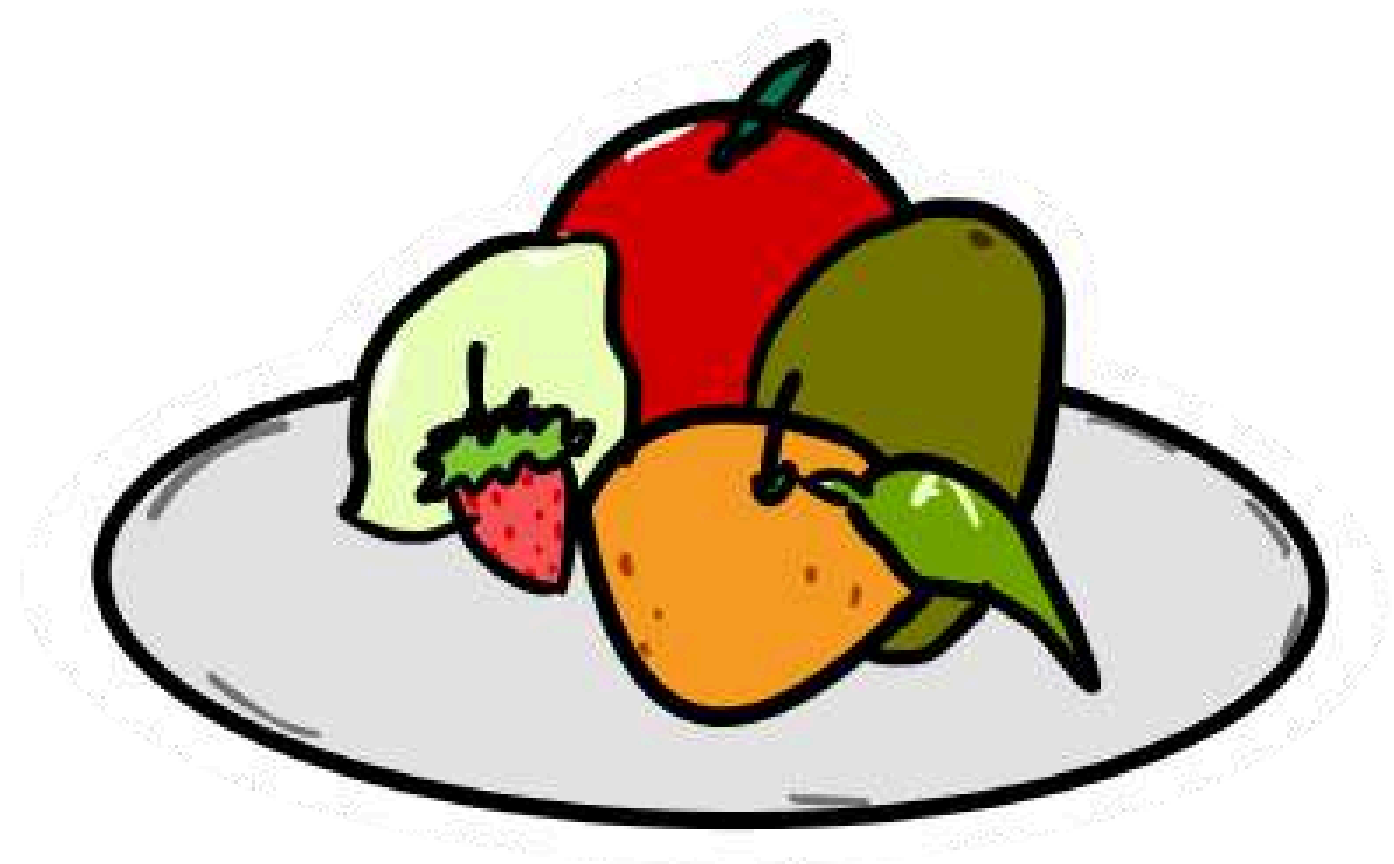


程式說明



重點項目

1. 基本運算程式碼 (bmi, 基礎代謝率, 目標熱量)
2. 介面程式碼 (tkinter, prettytable)
3. 比對食物相似度並找尋熱量程式碼
4. 運動抓資料程式碼
5. 運動推薦機制程式碼
6. 讀檔寫檔程式碼
7. 讀資料程式碼 (pandas...)



1. 基本運算程式碼

(BMI, 基礎代謝率, 目標熱量)

基本運算（BMI, 基礎代謝率, 目標熱量）

```
try:
    if sex_entry.get() != '0' and goal_entry.get() != '0' and activity_entry.get() != '0':
        height = float(height_entry.get())
        weight = float(weight_entry.get())
        age = float(age_entry.get())
        sex = sex_entry.get()
        goal = goal_entry.get()
        activity = activity_entry.get()

    bmi = weight / ((height / 100) ** 2)
    if bmi < 18.5:
        standard = "體重過輕"
    elif bmi < 24:
        standard = "體重適中"
    else:
        standard = "體重過重"

    if sex == '男':
        basic_kalorie = 66 + (13.7 * weight) + (5.0 * height) - (6.8 * age)
    else:
        basic_kalorie = 655 + (9.6 * weight) + (1.8 * height) - (4.7 * age)

    if goal == '減肥':
        goal_kalorie = 1.3 * basic_kalorie - 300
    elif goal == '維持':
        goal_kalorie = 1.3 * basic_kalorie
    else:
        goal_kalorie = 1.3 * basic_kalorie + 300
```

- 讀取個人資料頁面的各個項目後，先計算BMI(體重除以身高平方)以及體重等級
- 分別套用男女的基礎代謝率公式
- 如果目標是維持當前體重，其目標熱量約略等於基礎代謝率的1.3倍，而若要增肥或減肥則以此基準再增減目標熱量

2. 介面程式碼

(tkinter, prettytable)

- 程式說明 / 介面

```
import tkinter as tk
from tkinter import *
from tkinter import ttk
from tkinter.constants import CENTER
from prettytable import PrettyTable
from PIL import Image, ImageTk
import os
import base64
```

```
from 小人1.png import img as img
tmp1 = open('小人1.png', 'wb')
tmp1.write(base64.b64decode(img))
tmp1.close()
# ... 其他圖片相同(略)
```

```
window = tk.Tk()
window.title('經天一起動吃動')
window.geometry('800x400')
window.state("zoomed")
window.configure(background='#D6E1E3')
```

```
# .....
```

```
window.mainloop()
```

```
os.remove('小人1.png')
# ... 其他圖片相同(略)
```

使用**tkinter**製作介面，**prettytable**製作表格

並以**"window"**作為主視窗介面

先用**base64**將圖片轉成**py**檔

再用**PIL**插入圖片

最後用**os**來刪除圖片

第一頁（個人資料頁面）

```
def page_1():  
    for i in window.winfo_children():  
        i.destroy()  
    def calculate_kalorie():  
        # ... 運算(略)  
        try:  
            if sex_entry.get() != '0' and height_entry.get() != '' #其他條件亦同(略)  
                # ... 運算(略)  
        except:  
            window = tk.Tk()  
            window.title('Error')  
            window.geometry('400x300')  
            window.configure(background='white')  
            error1 = tk.Label(window, text="出現錯誤，請檢查以下可能情況:", bg="white")  
            error1.pack()  
            # ... 其他文字(略)  
            assert False  
  
    # 標題  
    header_label = tk.Label(window, text='個人資料', font=('微軟正黑體', 25, 'bold'), padx=200, bg='#7085A3', fg='white')  
    header_label.pack(fill='x', pady = 10)
```

進入第一頁時刪除使用者眼前畫面

使用者有輸入的情況下才跑運算，若沒輸入就按其他頁，則用存於txt檔的資料去跑

程式無法執行(第一次使用因此txt檔裡無資料，或是輸入了非數字之身高體重年齡)，則跳出錯誤新分頁並中止程式

Label做標題

第一頁（個人資料頁面）

```
# 身高輸入文字框（體重，年齡亦同）
height_frame = tk.Frame(window, bg = '#D6E1E3')
height_frame.place(x = 500, y = 180)
height_label = tk.Label(height_frame, text='身高 (cm)', font=('微軟正黑體', 15), bg = '#D6E1E3')
height_label.pack(side=tk.LEFT)
height_entry = tk.Entry(height_frame)
height_entry.pack(side=tk.LEFT)
# 性別單選（活動量，目標亦同）
sex_frame = tk.Frame(window)
sex_frame.place(x = 500, y = 330)
sex_label = tk.Label(sex_frame, text='性別', font=('微軟正黑體', 15), activeforeground='#f00', bg = '#D6E1E3')
sex_label.pack(side=tk.LEFT)
sex_entry = tk.StringVar()
sex_entry.set(0)
sex_1=tk.Radiobutton(sex_frame, text="男", variable=sex_entry, value="男", font=('微軟正黑體', 12), activeforeground='dodgerblue', bg = '#D6E1E3')
sex_2=tk.Radiobutton(sex_frame, text="女", variable=sex_entry, value="女", font=('微軟正黑體', 12), activeforeground='dodgerblue', bg = '#D6E1E3')
sex_1.pack(side=tk.LEFT)
sex_2.pack(side=tk.LEFT)
```

用Entry做文字框，Radiobutton做單選框

之後使用.get()取得使用者輸入的值

第一頁（個人資料頁面）

```
# 裝飾圖片
im2 = Image.open('五人.png')
w, h = im2.size
ww = int(w/4.4)
hh = int(h/4.4)
im2 = im2.resize((ww, hh), Image.ANTIALIAS)
photo = ImageTk.PhotoImage(im2)
label = tk.Label(image=photo, bg='#D6E1E3')
label.image = photo
label.place(relx=0.63, rely=0.64)

# 按鈕
page_1_button = tk.Button(window, command=lambda:[calculate_kalorie(),page_2()], text="分析結果頁面", font=('微軟正黑體', 15),
                           activeforeground='dodgerblue', bg='white')
page_1_button.place(x = 100, y = 240)

# ...跳置其他頁按鈕亦同(略)
```

用.open()開啟圖片、PhotoImage插入圖片，並以Button製作按鈕

按下按鈕就要跑一次calculate_kalorie()，使換頁前能計算出要取得的數據，再跑到欲前往頁面

第二頁（分析結果頁面）

```
def page_2():
    for i in window.winfo_children():
        i.destroy()
    def sentence():
        # ... 第二頁要呈現的東西(略)

    # 第二頁的標題
    header_label = tk.Label(window, text='分析結果頁面', font=('微軟正黑體', 25, 'bold'), bg='#7085A3', fg='white', padx=200)
    header_label.pack(fill='x', pady = 10)

    # 第二頁呈現
    result_1_label = tk.Label(window)
    result_1_label.place(x = 470, y = 240)
    # ... 其他亦同(略)

    page_1_button = tk.Button(window, command=page_1, text="個人資料頁面", font=('微軟正黑體', 15), activeforeground='dodgerblue', bg='white')
    page_1_button.place(x = 100, y = 180)
    # ... 跳置其他頁按鈕亦同(略)
```

標題、欲呈現文字皆已用Label製作

按下按鈕則跑到欲前往頁面

第三之一頁（食物紀錄頁面）

```
def page_3_1():
    for i in window.winfo_children():
        i.destroy()
    def save_input():
        food_type = food_type_entry.get()
        deal = deal_entry.get()
        if deal == '0' or food_type == '':
            # ... 錯誤頁面與前面提到製作方式相同(略)

        else:
            if len(intake_kalorie_entry.get()) == 0:
                # ... 食物資料庫篩選機制略

            if len(similar_num_lst) != 0:
                continue_button = tk.Button(window, command=lambda:[save_input2(), page_3_1()], text="繼續輸入", font=('微軟正黑體', 15),
                                              activeforeground='dodgerblue')
                continue_button.pack(pady = 10)
            elif len(similar_num_lst) == 0:
                # ... 略

            elif len(ka_lst) == 0:
                again_button = tk.Button(window, text="偵測不到相關食物，請重新輸入", command=page_3_1, font=('微軟正黑體', 15),
                                          activeforeground='dodgerblue', bg='white')
                again_button.pack(pady = 10)
```

若使用者未輸入餐期、食物項目，將跳出錯誤頁面並中止程式

若能篩出相似項目則跑出"繼續輸入"按鈕，按下則跑一次save_input2()，使各個分類list中取得紀錄，再重新執行第3-1頁(此頁)

篩不出相似項目則跑出"偵測不到..."按鈕，按下則重跑一次此頁

第三之一頁（食物紀錄頁面）

```
if intake_kalorie_entry.get() != '':
    try:
        intake_kalorie = float(intake_kalorie_entry.get())
        continue_button = tk.Button(window, command=lambda:[save_input2(), page_3_1()], text="繼續輸入", font=('微軟正黑體', 15),
                                     activeforeground='dodgerblue')

        continue_button.pack(pady = 10)
    except:

        # ... 錯誤頁面與前面提到製作方式相同(略)

def save_input2():

    # 在各個分類list中存入紀錄過程(略)
```

若使用者自己輸入且輸入正確則跑出"繼續輸入按鈕"(執行與前面提到同)

若程式無法正常執行(輸入非數字熱量/讀不到相似食物資料仍執行其他按鈕)，
則跑出錯誤視窗並中止程式

第三之一頁（食物紀錄頁面）

```
# 標題、輸入框、單選框(與前面提到一樣處略)
header_label_food = tk.Label
food_type_entry = tk.Entry
deal_1 = tk.Radiobutton
intake_kalorie_entry = tk.Entry

# "ok按鈕"
ok_button = tk.Button(window, text="ok", command=save_input, font=('微軟正黑體', 15),
                        activeforeground='dodgerblue', bg='white')
ok_button.pack(side=tk.TOP, pady = 10)

# 切換頁面按鈕()
page_1_button = tk.Button(window, command=lambda:[save_input2(), page_3_1(), page_1()], text="個人資料頁面", font=('微軟正黑體', 15),
                           activeforeground='dodgerblue', bg='white')
page_1_button.place(x = 100, y = 180)
# ...跳置其他頁按鈕亦同(略)

# 圖片(與前面提到一樣處略)
im = Image.open('水果盤.png')
photo = ImageTk.PhotoImage(im)

im2 = Image.open('珍奶.png')
photo = ImageTk.PhotoImage(im2)
```

呈現方式與前面提到的相同，

按下ok按鈕則跑一次save_input()，跑出繼續輸入或是偵測不到按鈕

按下切換頁面按鈕則要跑一次save_input2()與page_3_1儲存需要的資訊，再跑到欲前往頁面

第三之二頁（運動紀錄頁面）

```
# 第三頁(運動)
def page_3_2():
    for i in window.winfo_children():
        i.destroy()
    def select_emerge():
        try:
            # 運動抓資料過程(略)

            if len(item) != 0:
                exercise_select = ttk.Combobox(window, values = item, font=('微軟正黑體',15))
                exercise_select.pack(pady = 10)
                continue_button = tk.Button(window, command=lambda:[save_input(), page_3_2()], text="繼續輸入", activeforeground='dodgerblue', font=('微軟正黑體',
                continue_button.pack(pady = 10)
                re_button = tk.Button(window, text="重填運動項目/時間", command=page_3_2, activeforeground='dodgerblue', font=('微軟正黑體',15), bg='white')
                re_button.pack(pady = 10)

            else:
                again_button = tk.Button(window, text="偵測不到相關運動，請重新輸入", command=page_3_2, activeforeground='dodgerblue', font=('微軟正黑體',15), bg=
                again_button.pack(pady = 10)

        except:
            # ... 錯誤頁面與前面提到製作方式相同(略)

    def save_input():
        try:
            a = exercise_select.get()

        except:
            a = '!'
```

偵測到運動項目，則用Combobox做出一個下拉式選單供使用者做選擇，並跳出繼續輸入按鈕(按下則在list中記錄資料並重跑此頁)或是重填按鈕(按下則重跑此頁)

偵測不到則跑出按鈕，按下則重跑此頁

第三之二頁（運動紀錄頁面）

```
# 其他情況list儲存運動資料過程略

elif a == '':

    # ... 錯誤頁面與前面提到製作方式相同(略)

# 標題、文字框(與前面提到一樣處略)
header_label_exercise = tk.Label
exercise_type_entry = tk.Entry
consume_time_entry = tk.Entry

# “選擇強度/類型”按鈕
select_button = tk.Button(window, text="選擇強度/類型", command=lambda:[save_input(),select_emerge()], font=('微軟正黑體',15),bg='white',activeforeground='dodgerblue')
select_button.pack(pady=10)

# 切換頁面按鈕
page_1_button = tk.Button(window, command=lambda:[save_input(), page_1()], text="個人資料頁面", font=('微軟正黑體',15),bg='white',
                           activeforeground='dodgerblue')
page_1_button.place(x = 100, y = 180)
# ... 跳置其他頁按鈕亦同(略)

# 圖片(與前面提到一樣處略)
im = Image.open('小人1.png')
im = Image.open('小人2.png')
im = Image.open('小人3.png')
```

若點選強度鈕後未做選擇就按任何按鈕，會得到下拉選單預設的' '值而跑出錯誤視窗，並中止程式

其他呈現方式與前面提到相同

按下"選擇強度/類型"按鈕，就跑一次save_input(), select_emerge()，存下使用者輸入的資料，並跑出下拉選單與按鈕

按下切換頁面按鈕則跑一次save_input()儲存需要的資訊，再跑到欲前往頁面

第四頁（數據總結頁面）

```
class VerticalScrolledFrame(ttk.Frame):
    def __init__(self, parent, *args, **kw):
        def _configure_interior(event):
            size = (interior.winfo_reqwidth(), interior.winfo_reqheight())
            canvas.config(scrollregion="0 0 %s %s" % size)
            if interior.winfo_reqwidth() != canvas.winfo_width():
                canvas.config(width=interior.winfo_reqwidth())
        def _configure_canvas(event):
            if interior.winfo_reqwidth() != canvas.winfo_width():
                # update the inner frame's width to fill the canvas
                canvas.itemconfigure(interior_id, width=canvas.winfo_width())
        def _on_mousewheel(event, scroll):
            canvas.yview_scroll(int(scroll), "units")
        def _bind_to_mousewheel(event):
            canvas.bind_all("<Button-4>", fp(_on_mousewheel, scroll=-1))
            canvas.bind_all("<Button-5>", fp(_on_mousewheel, scroll=1))
        def _unbind_from_mousewheel(event):
            canvas.unbind_all("<Button-4>")
            canvas.unbind_all("<Button-5>")

        ttk.Frame.__init__(self, parent, *args, **kw)
        vscrollbar = ttk.Scrollbar(self, orient=tk.VERTICAL)
        vscrollbar.pack(fill=tk.Y, side=tk.RIGHT, expand=tk.FALSE)
        canvas = tk.Canvas(self, bd=0, highlightthickness=0, bg='#D6E1E3', yscrollcommand=vscrollbar.set)
        canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=tk.TRUE)
        vscrollbar.config(command=canvas.yview)
        canvas.xview_moveto(0)
        canvas.yview_moveto(0)
        self.interior = interior = Frame(canvas, bg='#D6E1E3')
        interior_id = canvas.create_window(0, 0, window=interior, anchor=tk.NW)
        interior.bind('<Configure>', _configure_interior)
        canvas.bind('<Configure>', _configure_canvas)
        canvas.bind('<Enter>', _bind_to_mousewheel)
        canvas.bind('<Leave>', _unbind_from_mousewheel)
```

用Scrollbar做第四頁滾動軸
的前置作業

第四頁（數據總結頁面）

```
def page_4():
    for i in window.winfo_children():
        i.destroy()

    # 滾動軸
    if __name__ == "__main__":
        holder_frame = tk.Frame(window)
        holder_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=tk.TRUE)

        second_frame = VerticalScrolledFrame(holder_frame)
        second_frame
        second_frame.pack(side=tk.BOTTOM, fill=tk.BOTH, expand=tk.TRUE)

    # ... 熱量計算(略)

    # 標題、提醒的話、細項文字呈現(與前面提到一樣處略)
    header_label_1 = tk.Label
    if remaining_kalorie > 0:
        result_2_label = tk.Label      # ... 其他剩餘熱量情況略
    result_3_label = tk.Label
```

- 放入滾動軸
- 其他呈現方式與前面提到的相同

第四頁（數據總結頁面）

```
# 表格製作與呈現
breakfast_table = PrettyTable()
breakfast_table.add_column('攝取食物', total_food_lst)
breakfast_table.add_column('熱量', total_kcal_lst)
result_5 = tk.Text(second_frame.interior, bd=0, bg='#D6E1E3', width = 50, height = len(total_kcal_lst)+4)
result_5.tag_configure("tag_name", justify='center')
result_5.insert('1.0', breakfast_table)
result_5.tag_add("tag_name", "1.0", "end")
result_5.configure(state='disabled')
result_5.pack(side="left")

# ... 運動表格與食物表格製作方式相同(略)

# 切換頁面按鈕
page_1_button = tk.Button(window, text="個人資料頁面", command=page_1, font=('微軟正黑體', 15),
                           activeforeground='dodgerblue', bg='white')
page_1_button.place(x = 100, y = 180)
# ... 跳置其他頁按鈕亦同(略)

# 圖片(與前面提到一樣處略)
im2 = Image.open('披薩.png')
im2 = Image.open('蛋糕1.png')
```

- 用PrettyTable製作食物與運動兩個表格，並把表格嵌入Text文字框中
- 按下切換頁面按鈕直接跑至欲前往頁面
- 圖片呈現方式與前面提到相同

第四頁（數據總結頁面）

```
# "運動推薦"按鈕
recommend_button = tk.Button(window, text="運動推薦", command=page_5, font=('微軟正黑體', 15), activeforeground='dodgerblue', bg='white')
recommend_button.place(x = 1050, y = 360)
# "關閉重新啟動"
close_button = tk.Button(window, command = lambda : [window.destroy(), close()], text="關閉重新啟動", font=('微軟正黑體', 15), activeforeground='dodge:
close_button.place(x = 1050, y = 180)
# "刪除特定紀錄"按鈕
delete_button = tk.Button(window, text="刪除特定紀錄", command=delete, font=('微軟正黑體', 15), activeforeground='dodgerblue', bg='white')
delete_button.place(x = 1050, y = 240)
# "刪除所有記錄"按鈕
restart_button = tk.Button(window, text="刪除所有記錄", command = restart, font=('微軟正黑體', 15), activeforeground='dodgerblue', bg='white')
restart_button.place(x = 1050, y = 300)
```

```
def restart():
    breakfast_type_list = []
    breakfast_kalorie_list = []

    # ... 其他list操作模式相同(略)

    page_4()
```

```
def close():
    with open(r'donchidon.txt', 'w') as donchidon:
        donchidon.truncate(0)
```

按下運動推薦按鈕與刪除特定紀錄按鈕則跑到欲到達的新視窗(下頁說明)

按下關閉重新啟動則跑一次close()，用.truncate(0)刪除txt檔所有資料
並用.destroy()自動關閉視窗

按下刪除所有紀錄則跑一次restart()，讓所有list的內容刪除

運動推薦頁面

```
def page_5():
    window = tk.Tk()
    window.title('運動推薦')
    window.geometry('800x600')
    window.configure(background='#D6E1E3')
    def recommendation():

        # ...運動推薦機制過程略

    def open_url(event):
        webbrowser.open(result_6)

        # ...其他推薦影片亦同(略)

    result_label = tk.Label #...與前面提到相同處略
    u2 = tk.Button(window, bg='white')
    u2.pack(pady=3)
    u2.bind("<Button-1>", open_web)

    # ...其他呈現方式均同(略)

    recommendation()
```

開一個新視窗

要呈現的文字皆以Label呈現

用webbrowser.open(), Button, .bind()

製作點擊按鈕開啟影片連結的功能

刪除特定紀錄頁面

```
def delete():  
    window2 = tk.Toplevel(window)  
    window2.title('刪除特定紀錄')  
    window2.geometry('400x300')  
    window2.configure(background='#D6E1E3')  
    def delete2():  
        window2.destroy()  
        delete()  
    def combobox_emerge():  
        global food_select  
        if deal_entry.get() == '早餐':  
            if len(breakfast_type_list) == 0:  
                again_button = tk.Button(window2, text="無記錄資料，請重新選擇", command = delete2, font=('微軟正黑體', 12),  
                    activeforeground='dodgerblue', bg='white')  
  
                again_button.pack(pady = 6)  
            else:  
                food_select = ttk.Combobox(window2, values = list(set(breakfast_type_list)), font=('微軟正黑體', 12))  
                food_select.pack(pady = 6)  
                sure_button = tk.Button(window2, text="確定刪除", command = change_save_input, font=('微軟正黑體', 12),  
                    activeforeground='dodgerblue', bg='white')  
  
                sure_button.pack(pady = 6)  
                re_button = tk.Button(window2, text="重新選擇", command = delete2, font=('微軟正黑體', 12),  
                    activeforeground='dodgerblue', bg='white')  
  
                re_button.pack(pady = 6)  
  
# ... 午餐晚餐其他運動均同(略)
```

用.Toplevel()開一個新視窗

根據欲刪除的餐期，若沒有紀錄則跳出"無記錄..."按鈕，
按下後重跑此畫面

若有紀錄則跳出下拉選單、確定刪除按鈕(按下後進入change_input()
進行刪除)及重新選擇按鈕(按下後則重跑此畫面)

刪除特定紀錄頁面

```
if deal_entry.get() == '0':  
    # ... 錯誤畫面與前面提到相同(略)  
  
def change_save_input():  
    try:  
        a = food_select.get()  
    except:  
        a = '!'  
  
    if a == '':  
        # ... 錯誤畫面與前面提到相同(略)  
  
    elif a == '!':  
        breakfast_type_list = breakfast_type_list  
        breakfast_kalorie_list = breakfast_kalorie_list  
  
        # ... 午餐晚餐其他運動均同(略)
```

若沒在下拉選單中做選擇，則會跑出錯誤視窗並中止程式

刪除特定紀錄頁面

```
else:
    if deal_entry.get() == '早餐':
        if len(breakfast_type_list) != len(set(breakfast_type_list)):
            def a():
                try:
                    row = int(row_entry.get()) - 1
                    breakfast_kalorie_list.pop(row)
                    breakfast_type_list.pop(row)
                    page_4()
                except:
                    # ... 錯誤畫面與前面提到相同(略)
            # 輸入刪除第幾行文字框(前面提到處略)
            row_entry = tk.Entry
            button = tk.Button(window2, text="確定", command = a, font=('微軟正黑體', 12),
                               activeforeground='dodgerblue', bg='white')
            button.pack(pady = 6)
        else:
            # list中移除項目略

# ... 午餐晚餐其他運動均同(略)

# 單選框(前面提到處略)
deal_1 = tk.Radiobutton(

select_button = tk.Button(window2, text="選擇要刪除的食物/運動種類", command = lambda:[change_save_input(), combobox_emerge()], font=('微軟正黑體', 12),
                           activeforeground='dodgerblue', bg='white')
select_button.pack(pady = 6)
```

若行數輸入不正確導致程式無法跑，則會跑出錯誤視窗，並中止程序

當有重複記錄時，會讓使用者選擇刪除第幾行記錄的文字框

按下"選擇"...按鈕則跑一次change_save_input()及combobox_emerge()，取得需要的數據，並跑出下拉框與按鈕

3. 比對食物相似度並找尋熱量程式碼

食物熱量搜尋

```
# 若沒有輸入熱量則搜尋資料庫
if len(intake_kalorie_entry.get()) == 0:
    similar_food_lst = []
    intake_kalorie = 0

    id = '1RSBV_39b2U9W-0bmn_sJ4M2oAEbieTlq'
    url= f"https://docs.google.com/uc?id={id}&export=download"
    food_kalorie_df = pd.read_csv(url)

    for food in food_kalorie_df["食物"]:
        if food == food_type:
            ka_filter = food_kalorie_df["食物"] == food
            kaka = food_kalorie_df[ka_filter]["熱量"]
            for i in kaka:
                intake_kalorie = i
        elif (food in food_type_entry.get()) or (food_type_entry.get() in food):
            similar_food_lst.append(food)
```

**第一層找尋機制是找「完全包含」之食物名稱
並記錄到相似食物之list裡面**

若使用者沒有輸入熱量，則由我們根據使用者之輸入，來查找資料庫中之熱量，若有完全相符之名稱，則直接取之熱量來記錄。同時間為第一層找尋機制做準備，找出資料庫內含有輸入食物名，或輸入食物名中含有資料庫食物名稱的食物名稱，並予以紀錄。

當找不到名稱完全相符之食物名稱，則開始第一層找尋相似食物之機制。

相似度計算

```
if intake_kalorie == 0:
    similar_num_lst = []
    for food in similar_food_lst:
        if len(food_type_entry.get()) < len(food):
            similarity = len(food_type_entry.get()) / len(food)
        else:
            similarity = len(food) / len(food_type_entry.get())
        similar_num_lst.append(similarity)

    if len(similar_num_lst) != 0:
        similar_food = similar_food_lst[similar_num_lst.index(max(similar_num_lst))]
        ka_filter = food_kalorie_df["食物"] == similar_food
        kaka = food_kalorie_df[ka_filter]["熱量"]
        for i in kaka:
            intake_kalorie = i
# 做出"繼續輸入"按鈕，若按下則繼續輸入食物之熱量。若找不到完全包含的食物，則開啟第二層找尋機制。
continue_button = tk.Button(window, text="繼續輸入",
                                activeforeground='dodgerblue')

continue_button.pack(pady = 10)
```

根據list裡面的食物字數長度來計算相似度，若輸入食物名之字數多於相似食物之字數，則將輸入食物名之字數當分母，相似食物當分子，計算重複字數之比例；反之，則分母分子相反。

計算完所有相似食物之相似度後，找出相似度最大的食物，並找出其熱量當作輸入食物之熱量。若找不到完全包含的食物，則開啟第二層找尋機制。

舉例來說，若使用者輸入○拌麵，而資料庫只有拌麵之熱量，因「拌麵」包含於「○拌麵」，所以會被記錄為相似食物。而○拌麵字數大於拌麵，所以 $\text{len}(\text{"○拌麵"})$ 為分母， $\text{len}(\text{"拌麵"})$ 為分子，相似度為 $2/3$ 。

4. 運動抓資料程式碼

- 程式說明 / 運動抓資料

```
try:
    id = '1p7P8ZtxVsdK9kZ62xN7KvJg02-aXh3zL'
    url= f"https://docs.google.com/uc?id={id}&export=download"
    exercise_kalorie_df = pd.read_csv(url)

    exercise_type = exercise_type_entry.get()
    consume_time = float(consume_time_entry.get())

    item = []
    female_kal = []
    male_kal = []
    find_exer_lst = ""
    end_consume_kal_lst = []

    for exer in exercise_kalorie_df["運動項目"]:
        if exer == exercise_type:
            f = exercise_kalorie_df[exercise_kalorie_df["運動項目"] == exer]
            for q in f["運動項目"]:
                item.append(q)
            for vv in f["熱量(女)"]:
                female_kal.append(vv)
            for v in f["熱量(男)"]:
                male_kal.append(v)
        elif (str(exer) in str(exercise_type)) or (str(exercise_type) in str(exer)):
            f = exercise_kalorie_df[exercise_kalorie_df["運動項目"] == exer]
            for g in f["運動項目"]:
                item.append(g)
            for k in f["熱量(女)"]:
                female_kal.append(k)
            for h in f["熱量(男)"]:
                male_kal.append(h)
```

首先先讀取建立好的csv檔，裡面包含運動項目、熱量(女)和熱量(男)共三欄，而消耗熱量是以分鐘為計算單位。item是使用者輸入的運動項目，無論使用者輸入的關鍵字在運動項目欄，或是運動項目欄在使用者輸入的關鍵字裡，我們都以下拉選單呈現數據。

運動項目	熱量(男)	熱量(女)
跑步，每公里耗時3分鐘26秒	21	17
跑步，每公里耗時4分鐘9秒	17	14
跑步，每公里耗時5分鐘	14	11
跑步，每公里耗時7.5分鐘	9	8
跑步，慢跑/散步組合	7	6

- 程式說明 / 運動抓資料

```
if sex == "男":
    for p in range(0, len(item)):
        end_consume_kal = male_kal[p] * consume_time
        end_consume_kal_lst.append(end_consume_kal)
else:
    for c in range(0, len(item)):
        end_consume_kal = female_kal[c] * consume_time
        end_consume_kal_lst.append(end_consume_kal)
if len(item) != 0:
    # 做出下拉選單
    exercise_select = ttk.Combobox(window, values = item, font=('微軟正黑體',15))
    exercise_select.pack(pady = 10)
    # 做出"繼續輸入"按鈕，若按下則繼續儲存輸入資訊並停留在第3-2頁
    continue_button = tk.Button(window, command=lambda:[save_input(), page_3_2()],
    continue_button.pack(pady = 10)
    # 做出"重填運動項目/時間"按鈕，若按下則繼續儲存輸入資訊並停留在第3-2頁
    re_button = tk.Button(window, text="重填運動項目/時間", command=page_3_2, activebackground='red')
    re_button.pack(pady = 10)

else:
    # 做出"偵測不到相關運動，請重新輸入"按鈕，若按下則回到第3_2頁
    again_button = tk.Button(window, text="偵測不到相關運動，請重新輸入", command=page_3_2, activebackground='red')
    again_button.pack(pady = 10)
```

接著讀取使用者在個人資料頁面輸入的性別，在對應到熱量(女)或是熱量(男)的欄位後，乘以運動分鐘數，就會是這次運動消耗的熱量，最後再讀入數據總結頁面。

5. 運動推薦機制程式碼

讀取資料

```
def recommendation():  
    global result_6  
    global result_3  
    global result_9  
    id = '1p7P8ZtxVsdK9kZ62xN7KvJg02-aXh3zL'  
    url= f"https://docs.google.com/uc?id={id}&export=download"  
    df = pd.read_csv(url)  
    df.dropna(axis = 0, how = 'all', inplace = True)  
    men_kalorie = df['熱量(男)'][:61].astype(int)  
    women_kalorie = df['熱量(女)'][:61].astype(int)  
    df['影片連結'].fillna('沒有推薦影片', inplace = True)
```

利用建好的excel，用dataFrame紀錄運動時男性與女性每分鐘消耗的熱量以及推薦影片連結，若無連結，則放入'沒有推薦影片'。

隨機推薦運動項目

```
#男生的選擇
if sex == '男':
    index = -1
    light_for_men = []
    medium_for_men = []
    heavy_for_men = []
    for i in men_kalorie:
        index += 1
        if i <= 5:
            light_for_men.append(index)
        elif i >= 10:
            heavy_for_men.append(index)
        else:
            medium_for_men.append(index)
    if activity == '輕度':
        choice = random.choice(light_for_men)
        choice2 = random.choice(light_for_men)
        while choice == choice2:
            choice2 = random.choice(light_for_men)
    if activity == '中度':
        choice = random.choice(medium_for_men)
        choice2 = random.choice(medium_for_men)
        while choice == choice2:
            choice2 = random.choice(medium_for_men)
    if activity == '重度':
        choice = random.choice(heavy_for_men)
        choice2 = random.choice(heavy_for_men)
        while choice == choice2:
            choice2 = random.choice(heavy_for_men)
```

```
#帕梅拉或居家運動
choice3 = random.randint(122, len(df)-1)
while choice3 == choice2 or choice3 == choice:
    choice3 = random.randint(122, len(df)-1)
```

- 先區分性別，以男性為例
- 用每分鐘消耗的卡路里，區分輕度、中度與重度，並依照使用者選擇的強度，用 **random** 隨機選取推薦運動
- 從所有運動中挑選 **choice** 與 **choice2**
- 因我們希望輸出的運動中，至少能包含一項帕梅拉或居家運動，因此 **choice3** 會從帕梅拉與居家運動中挑選，三者若有重複就重新挑選

計算運動時長

```
#攝取過多熱量
if remaining_kalorie < 0:
    minutes = round(abs(remaining_kalorie)/df.loc[choice]['熱量(男)'])
    if minutes >= 60:
        hours = minutes // 60
        minutes = minutes - 60 * hours
        result_2 = f"您需要運動約 {hours} 小時 {minutes} 分鐘"
    else:
        result_2 = f"您需要運動約 {minutes} 分鐘"
minutes2 = round(abs(remaining_kalorie)/df.loc[choice2]['熱量(男)'])
if minutes2 >= 60:
    hours = minutes2 // 60
    minutes2 = minutes2 - 60 * hours
    result_5 = f"您需要運動約 {hours} 小時 {minutes2} 分鐘"
else:
    result_5 = f"您需要運動約 {minutes2} 分鐘"
minutes3 = round(abs(remaining_kalorie)/df.loc[choice3]['熱量(男)'])
if minutes3 >= 60:
    hours = minutes3 // 60
    minutes3 = minutes3 - 60 * hours
    result_8 = f"您需要運動約 {hours} 小時 {minutes3} 分鐘"
else:
    result_8 = f"您需要運動約 {minutes3} 分鐘"

#未攝取過多熱量
elif remaining_kalorie >= 0:
    result_2 = result_5 = result_8 = '運動時間自訂！'
```

若攝取過多熱量，利用推薦運動項目，以及每分鐘運動消耗的熱量，計算出需要運動的時長，超過六十分鐘則換算成小時。

若未攝取過多熱量，則不輸出運動時長，而是輸出「運動時間自訂」，使用者可自行選擇運動時長。

6. 讀檔寫檔程式碼

- 程式說明 / 讀檔寫檔

```
try:
    path="donchidon.txt"
    f=open(path,"r")
    data=f.readlines()
    f.close
    import re
    ye = data[0].split(":")[1][0:]
    bml = re.sub("\n", "", ye)
    bmi = float(bml)

    if bmi < 18.5:
        standard = "體重過輕"
    elif bmi < 24:
        standard = "體重適中"
    else:
        standard = "體重過重"

    ye3 = data[1].split(":")[1][0:]
    gk2 = re.sub("\n", "", ye3)
```

(以下省略

```
exercise_kalorie_list = ta[9]
page_4()

except:
    page_0()
```

執行程式時，會先試著開啟"donchidon.txt"，若能成功開啟檔案，則將文字檔中紀錄的數據內容彙整到程式內的各項變數中。因使用者不需重新輸入相關資訊，讀取檔案後將直接跳到第四頁。

若無法讀取檔案(可能是使用者第一次使用，或者重新啟動後)，則會開啟封面頁，引導使用者輸入相關資訊並記錄。

存檔

```
f=open("donchidon.txt","w")
f.write("bmi" + ":" + str(bmi) + "\n"
        "goal_kalorie"+":"+str(goal_kalorie)+"\n"+
        "basic_kalorie"+":"+str(basic_kalorie)+"\n"+
        "sex" + ":" + sex + "\n"
        "breakfast_type_list"+":"+str(breakfast_type_list)+"\n"+
        "breakfast_kalorie_list"+":"+str(breakfast_kalorie_list)+"\n"+
        "lunch_type_list"+":"+str(lunch_type_list)+"\n"+
        "lunch_kalorie_list"+":"+str(lunch_kalorie_list)+"\n"+
        "dinner_type_list"+":"+str(dinner_type_list)+"\n"+
        "dinner_kalorie_list"+":"+str(dinner_kalorie_list)+
        "\n"+"other_type_list"+":"+str(other_type_list)+"\n"+
        "other_kalorie_list"+":"+str(other_kalorie_list)+"\n"+
        "exercise_type_list"+":"+str(exercise_type_list)+"\n"+
        "exercise_kalorie_list"+":"+str(exercise_kalorie_list)+"\n"+
        "remaining_kalorie"+":"+str(remaining_kalorie)+"\n"+
        "height" + ":" + str(height) + "\n"
        "weight" + ":" + str(weight) + "\n"
        "age" + ":" + str(age) + "\n"
        "activity" + ":" + activity + "\n"
        "goal" + ":" + goal + "\n")
```

當進入第三頁時，會自動建立"donchidon.txt"紀錄包括身高、體重、性別等等的基本資料，以及運動項目、消耗熱量、食物紀錄、食物熱量等數據。若檔案已經存在，則會複寫原有的內容。

7. 讀資料程式碼 (pandas)

- 程式說明 / 讀資料

```
import pandas as pd
```

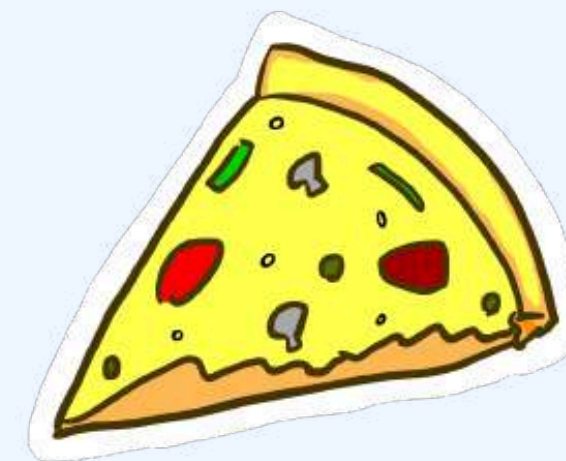
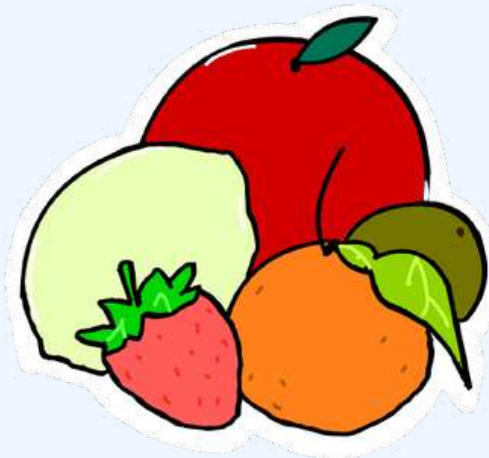
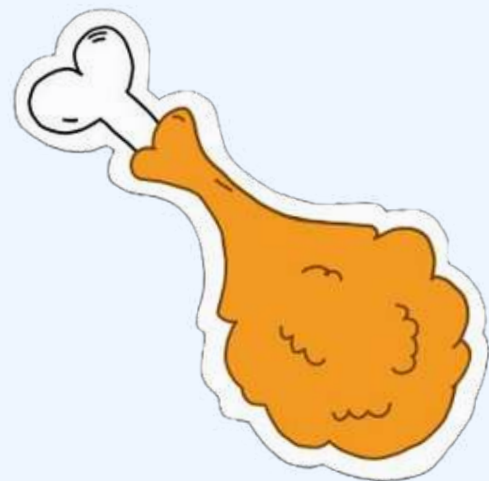
```
id = '1p7P8ZtxVsdK9kZ62xN7KvJgO2-aXh3zL'  
url= f"https://docs.google.com/uc?id={id}&export=download"  
df = pd.read_csv(url)
```

 https://drive.google.com/file/d/1p7P8ZtxVsdK9kZ62xN7KvJgO2-aXh3zL/view?usp=share_link

- 從雲端網址取得資料庫csv檔（藍線部分），並轉成url連結
- 用panda模組讀取資料

謝謝





DonChiDon

